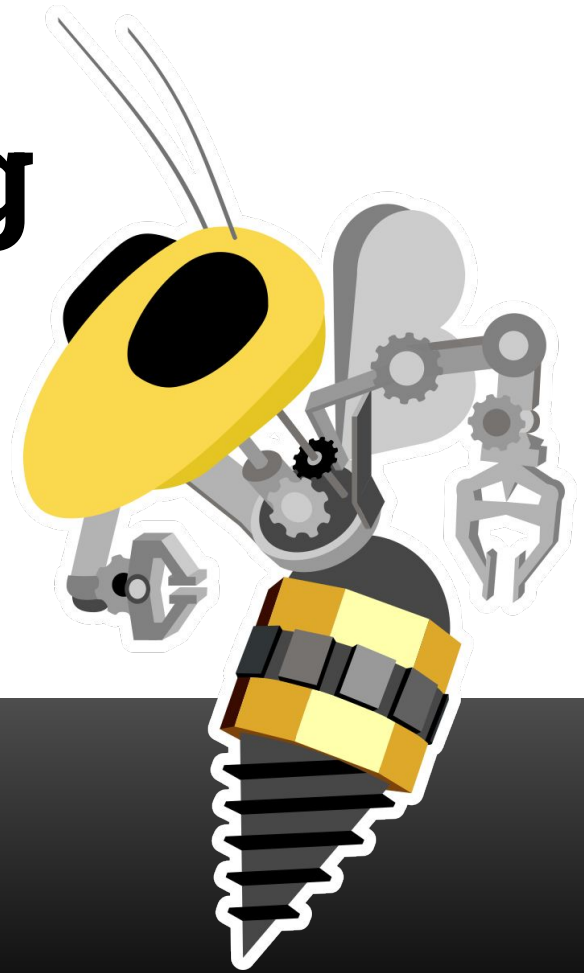


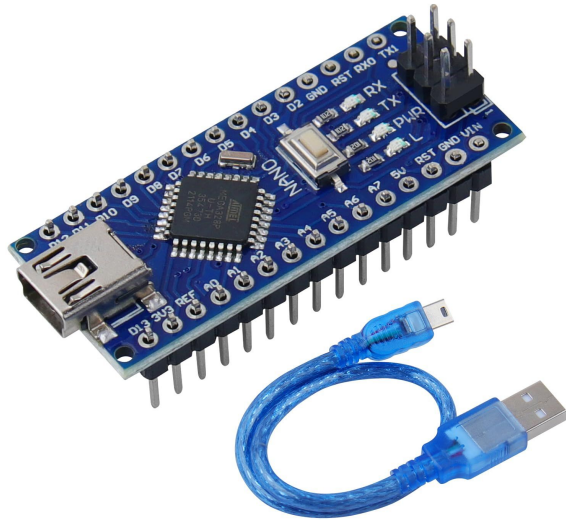
Firmware Training Week 3

Communication Protocols

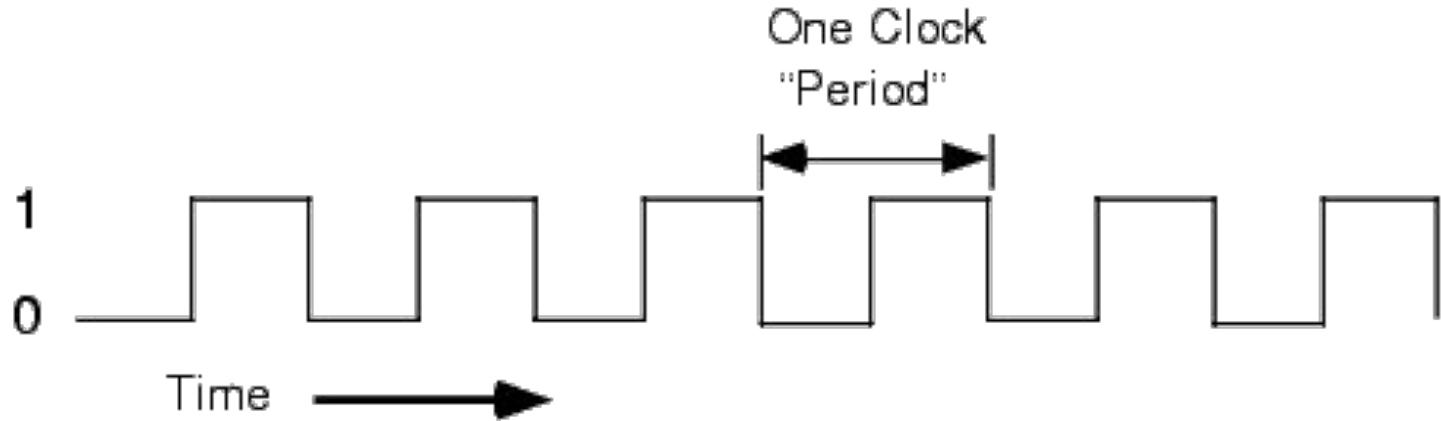


ROBOJACKETS
COMPETITIVE ROBOTICS AT GEORGIA TECH

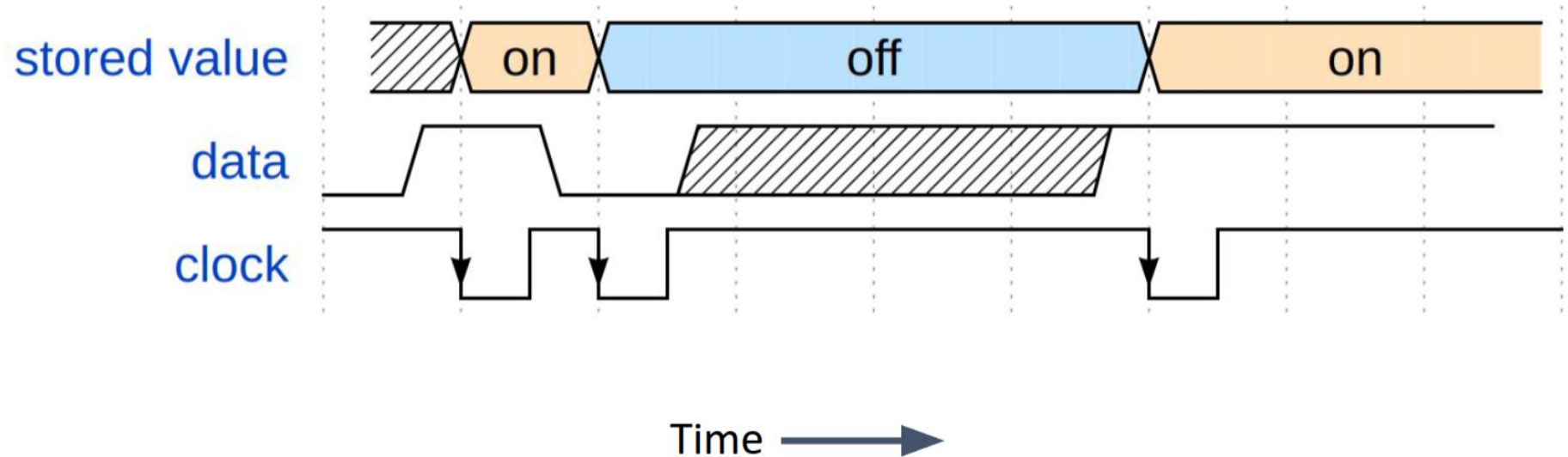
www.robojackets.org



Clocks



Clocks







Libraries

The screenshot displays the Arduino IDE Libraries Manager interface. The top bar shows the current project as 'main.cpp' and the active library as 'platformio.ini'. The left sidebar contains navigation icons for Home, Projects, Inspect, Libraries (selected), Boards, Platforms, and Devices. The main area is divided into several sections:

- Registry:** Includes tabs for 'Registry', 'Installed', 'Built-in', and 'Updates'. A search bar is present with the text 'Search libraries...'. Below the search bar, there are filters for 'tft display', 'dht*', 'header-RH_ASK.h', 'keyword:mqtt', 'framework:mbed', and 'platform:espressif8266', followed by a 'more...' link. Buttons for 'All Libraries' and 'Publish' are also visible.
- Recently:** A section showing recently added or updated libraries.
- Updated:** A list of libraries that have been updated, including 'Brasilino' (about an hour ago), 'jescore' (about an hour ago), 'ChronoLog' (3 hours ago), 'BLE-Gamepad-Client' (3 hours ago), and 'SHT4x' (3 hours ago).
- Added:** A list of libraries that have been added, including 'ChronoLog' (4 hours ago), 'PedroRobot' (9 hours ago), 'SFM3X00' (9 hours ago), 'ESPMeshMesh-dev' (19 hours ago), and 'HanumanMini' (1 day ago).
- Keywords:** A list of keywords used to filter libraries, including 'atmelavr', 'depth', 'encoder motor', 'esp32-idf', and 'aircrack'.
- Popular Tags:** A grid of tags representing popular library categories, including 'communication', 'sensors', 'control', 'device', 'display', 'esp32', 'other', 'data', 'arduino', 'input', 'i2c', 'signal', 'output', 'esp8266', 'sensor', 'processing', 'timing', 'lcd', 'iot', 'serial', 'wifi', 'tft', 'spi', 'temperature', 'storage', 'led', 'oled', 'graphics', 'mqtt', 'timer', 'ethernet', 'async', 'http', 'font', 'gclid', 'pwm', 'motor', 'carid', 'driver', 'stm32', 'library', 'button', 'webserver', 'interrupt', 'mbed', 'time', 'humidity', 'eeprom', 'rtc', and 'server'.
- Trending:** A section showing trending libraries, organized by 'Today', 'Week', and 'Month'.
 - Today:** AutoConnect, EmblUI, ArduinoWebsockets, RadioHead, PageBuilder, SimpleCollections, MD_Parola.
 - Week:** Adafruit BusIO, ArduinoJson, Adafruit GFX Library, Unity, Adafruit Unified Sensor, FastLED, Adafruit SSD1306.
 - Month:** Adafruit BusIO, Adafruit GFX Library, Adafruit TSL2561, Melopero RV3028, ArduinoJson, Adafruit Unified Sensor, Unity.

main.cpp

PIO Home

platformio.ini

< > ↩

Follow Us

📱

📺

🔄

Home

Registry

Installed

Built-in

Updates

RotaryEncoder

by Matthias Hertel

Use a rotary encoder with quadrature pulses as an input device. This library decodes the signals from a rotary encoder and translates them into a counter position. The library comes with some samples that show how to use the library with and without interrupts.

Installation

1.5.3 released 3 years ago

Add to Project

More info

Examples

Installation

Headers

Changelog

AcceleratedRotator

AcceleratedRotator

```
// -----
// AcceleratedRotator.h - Example for the RotaryEncoder library.
// This class is implemented for use with the Arduino environment.
// Copyright (c) by Matthias Hertel, http://www.mathertel.de
// This work is licensed under a BSD style license. See http://www.mathertel.de/License.aspx
// More information on: http://www.mathertel.de/Arduino
// -----
// 18.01.2014 created by Matthias Hertel
// 13.11.2019 converted to AcceleratedRotator by Damian Philipp
// 06.02.2021 conditions and settings added for ESP8266
// -----

// This example checks the state of the rotary encoder in the loop() function.
// It then computes an acceleration value and prints the current position when changed.
// There is detailed output given to the Serial.
// You may play around with the constants to fit to your needs.

// Hardware setup:
// Attach a rotary encoder with output pins to
// * 2 and 3 on Arduino UNO.
// * A2 and A3 can be used when directly using the ISR interrupts, see comments below.
// * D5 and D6 on ESP8266 board (e.g. NodeMCU).
// Swap the pins when direction is detected wrong.
// The common contact should be attached to ground.

#include <Arduino.h>
#include <RotaryEncoder.h>

#ifdef ARDUINO_AVR_UNO || defined(ARDUINO_AVR_NANO_EVERY)
// Example for Arduino UNO with input signals on pin 2 and 3
#define PIN_IN1 A2
#define PIN_IN2 A3

#elif defined(ESP8266)
// Example for ESP8266 NodeMCU with input signals on pin D5 and D6
#define PIN_IN1 D5
#define PIN_IN2 D6

#endif

#endif
```

Tags

signal

input

output

Platforms

Infineon XMC

Kindryte K210

Atmel AVR

Atmel SAM

Espressif 8266

Intel ARC32

Microchip PIC32

Nordic nRF91

ST STM32

Teensy

TI MSP430

TI TINA

Espressif 32

Nordic nRF52

ST STM8

Atmel megaAVR

Logic Green boards

Logirom OpenCPU Development Platform

Raspberry Pi RP2040

K1921VK

TI MSP432

Heltec CubeCell

LibreTiny

Renesas RA

Frameworks

Arduino

Authors

Matthias Hertel (maintainer)

Resources

Registry

Homepage

Repository

Downloads

7 in the last day

106 in the last week

587 in the last month

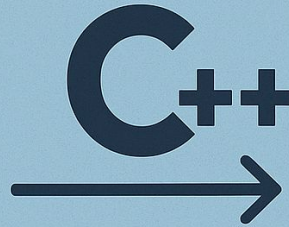
Versions

1.5.3 released 3 years ago

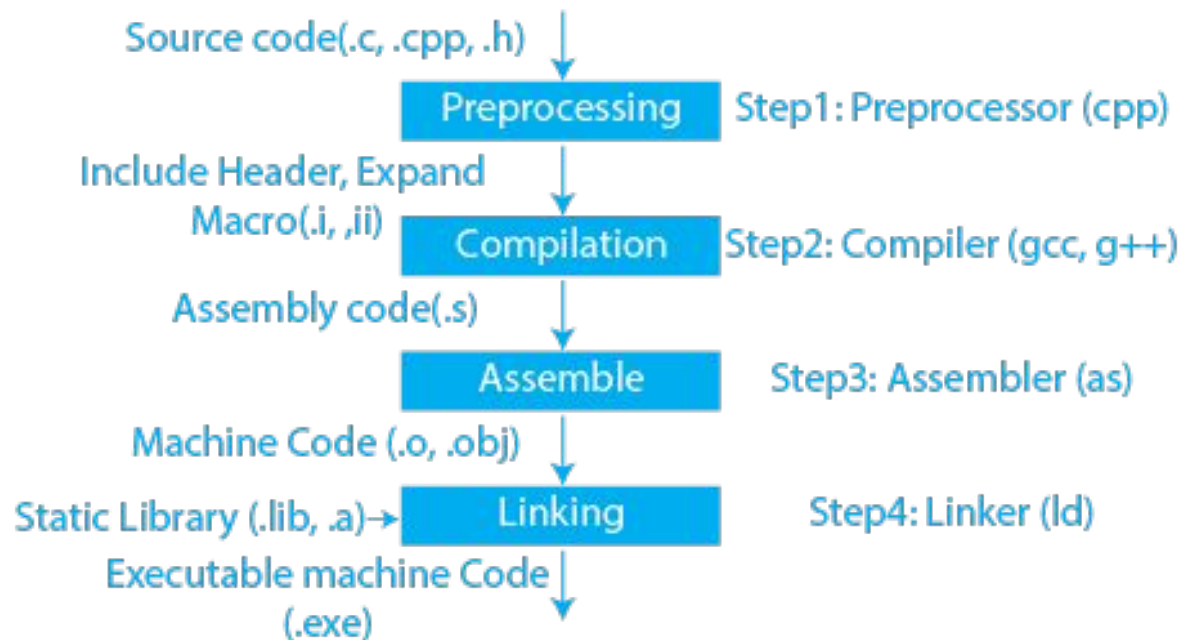
HEADER AND SOURCE FILES



header file



source file



```
1 // Including a system header
#include <Arduino.h>
#include <RotaryEncoder.h>

// Including a "local" dependency
#include "test.h"
#include "dinosaur.h"
```

EXPLORER

- BLINK
 - .pio
 - .vscode
 - include
 - header1.h
 - header2.h
 - header3.h
 - README
 - lib
 - src
 - header1.cpp
 - header2.cpp
 - header3.cpp
 - main.cpp
 - test
 - .gitignore
 - diagram.json
 - platformio.ini
 - README.md
 - wokwi.toml

main.cpp

```
src > main.cpp > ...
You, 14 seconds ago | 1 author (You)

1
2 // Including a system header
3 #include <Arduino.h>
4
5 #include "header1.h"
6
7 void setup() {
8     Header1Dinosaur d = Header1Dinosaur(10);
9 }
10
11 void loop() {
12
13 }
14
```

header1.h

```
include > header1.h > ...

1 #ifndef HEADER1_H
2 #define HEADER1_H
3
4 #include <Arduino.h>
5
6 class Header1Dinosaur {
7 public:
8     Header1Dinosaur(uint8_t data);
9
10 private:
11     uint8_t _data;
12 };
13
14 #endif
```

header1.cpp

```
src > header1.cpp > Header1Dinosaur(uint8_t)

1 #include "header1.h"
2
3 Header1Dinosaur::Header1Dinosaur(uint8_t data): _data(data)
4
5
```

Terminology

Master / Controller - The device starting the communication

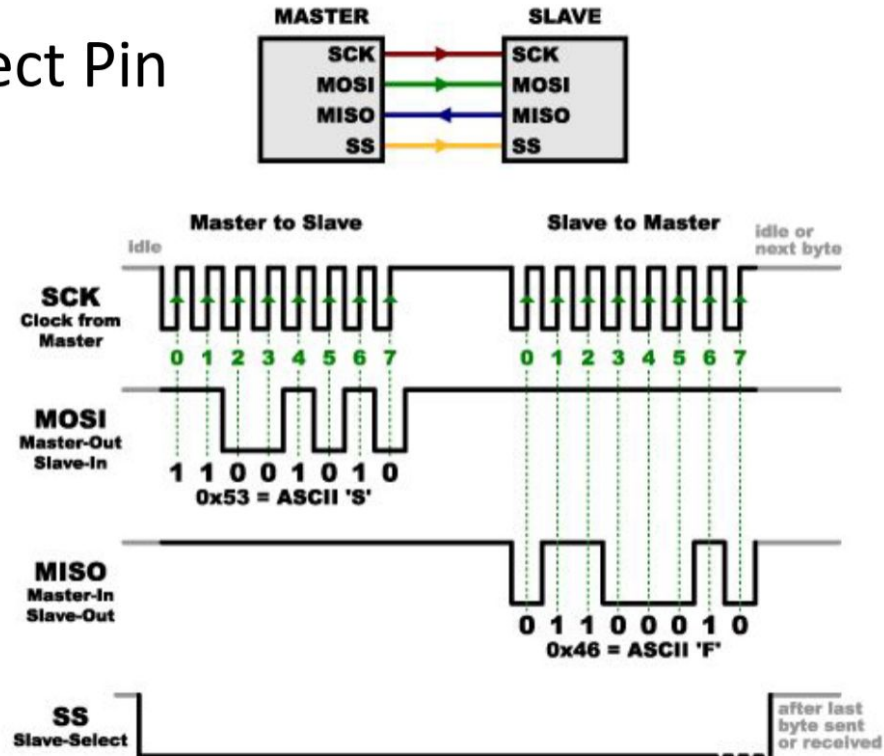
Slave / Peripheral - The device responding to the communication request

Full-Duplex - Transmit and Receive at same time

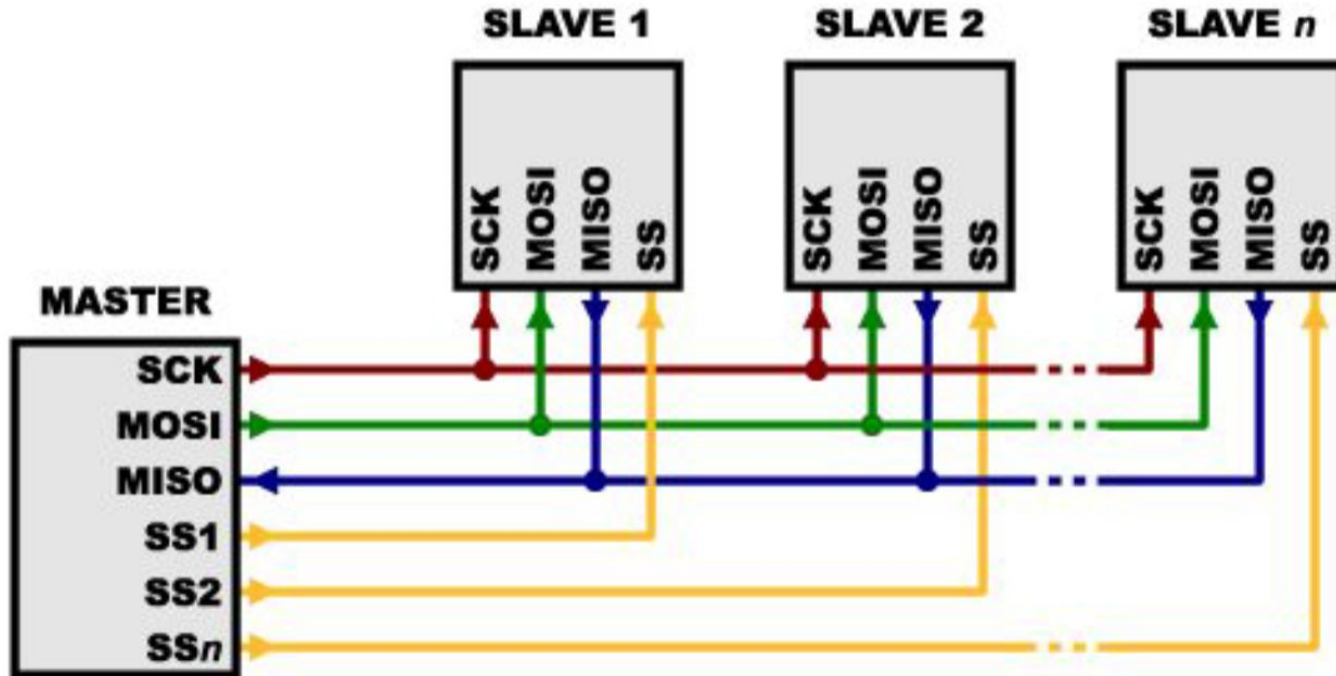
Half-Duplex - Only Transmit or Receive

SPI - Serial Peripheral Interface

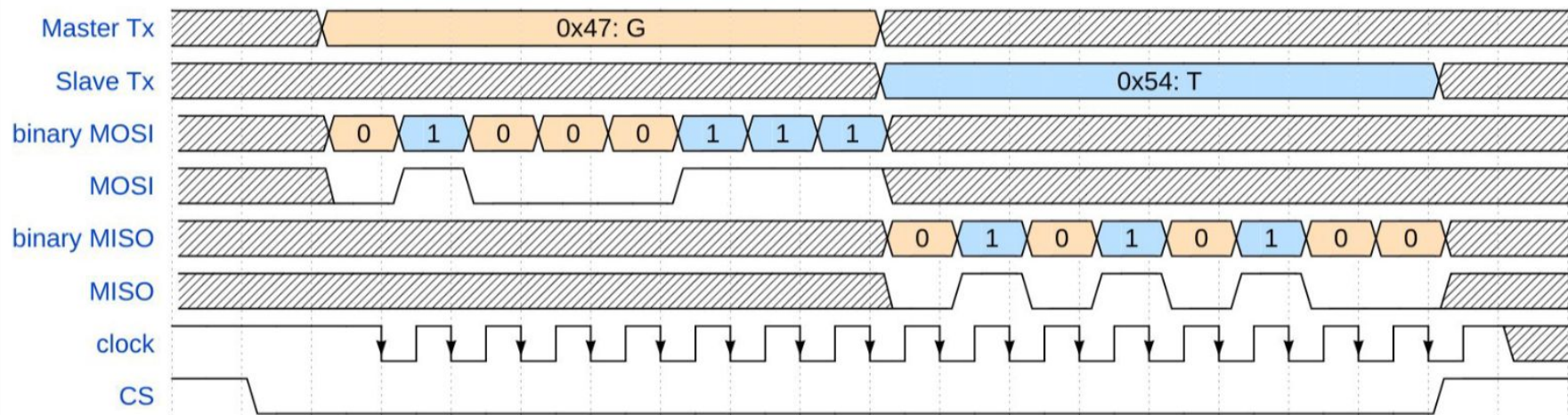
SPI – Select Pin



SPI – Single Master, Multiple Slaves



SPI Full Example



We send 'G' to the device, then the device sends 'T' back.

```
// Including a system header
#include <Arduino.h>
#include <SPI.h>

#define CIP0 12
#define COPI 11
#define SCLK 13
#define CSN 10

#define MAX_SPEED 1'000'000

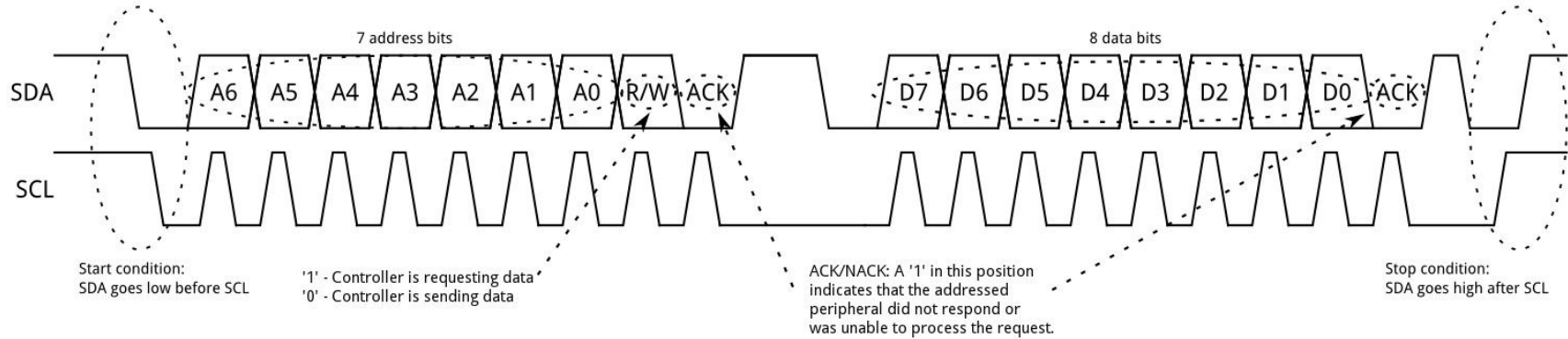
void setup() {
    pinMode(COPI, OUTPUT);
    pinMode(CIP0, INPUT);
    pinMode(SCLK, OUTPUT);
    pinMode(CSN, OUTPUT);
    digitalWrite(CSN, HIGH);
    SPI.begin();
}

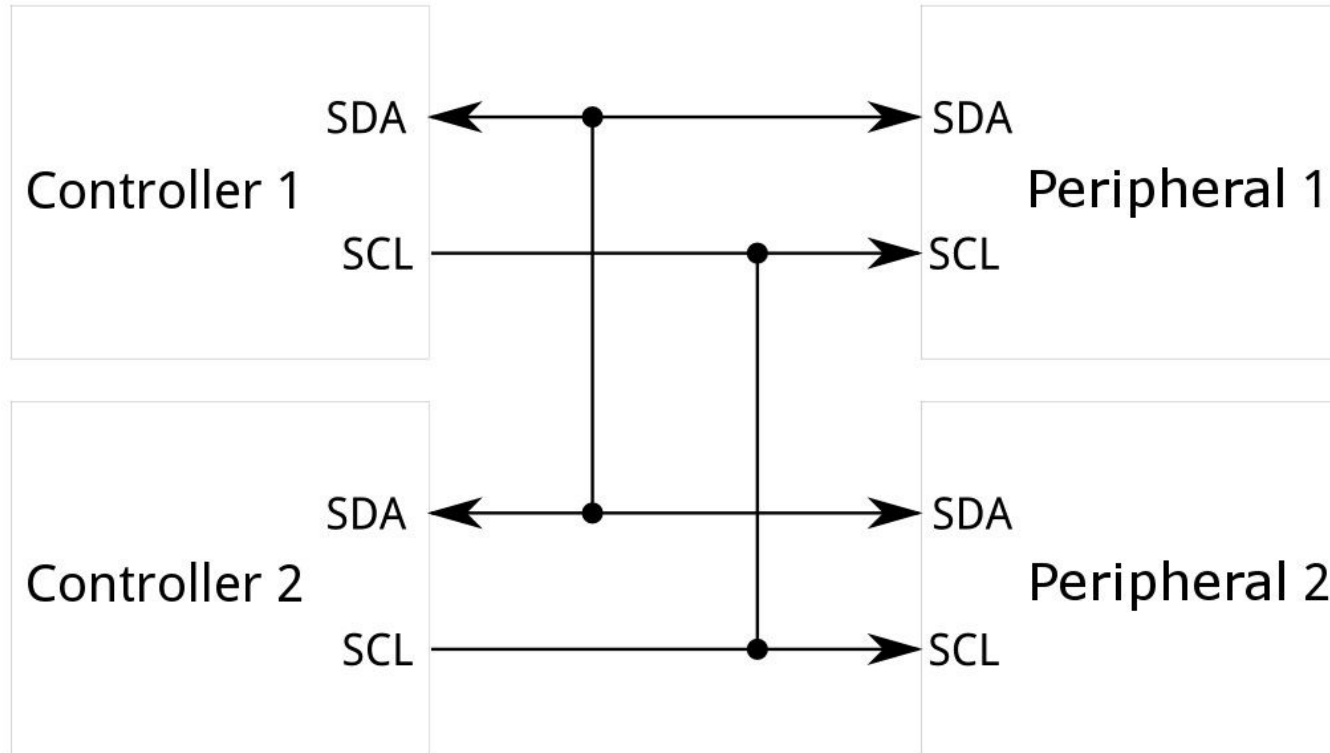
void loop() {
    // Start an SPI transaction at 1 MHz
    SPI.beginTransaction(SPISettings(1000000, MSBFIRST, SPI_MODE0));

    // Transfer (Full-Duplex) over SPI Peripheral
    uint8_t received = SPI.transfer(100);

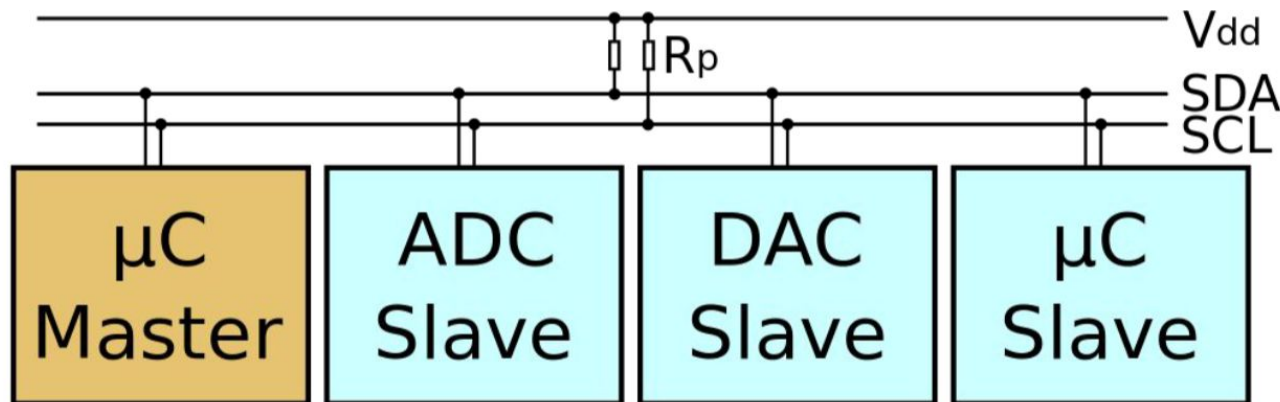
    // Stop an SPI transaction
    SPI.endTransaction();
}
```

I²C - Inter-Integrated Circuit





I2C – Pull-up Resistors



Pull-up resistors (R_p shown here) are used to keep the line high if no devices pull it low.
Typically 2.2KOhm

```
// Including a system header
#include <Arduino.h>
#include <Wire.h>

void setup() {
    Wire.begin();
}

void loop() {
    // Request 6 bytes from device ID 8
    Wire.requestFrom(8, 6);

    while (Wire.available()) {
        uint8_t data = Wire.read();
    }

    Wire.beginTransmission(8);

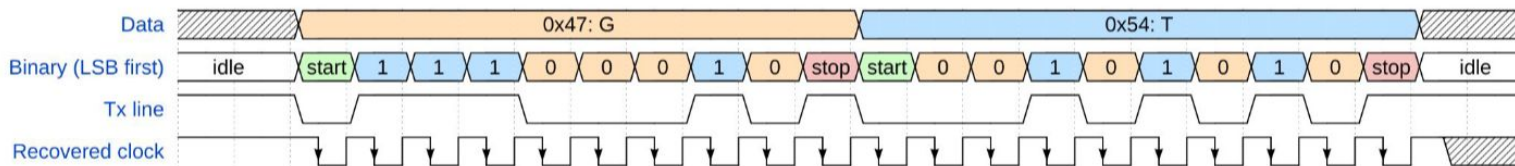
    Wire.write("test");

    Wire.endTransmission();
}
```

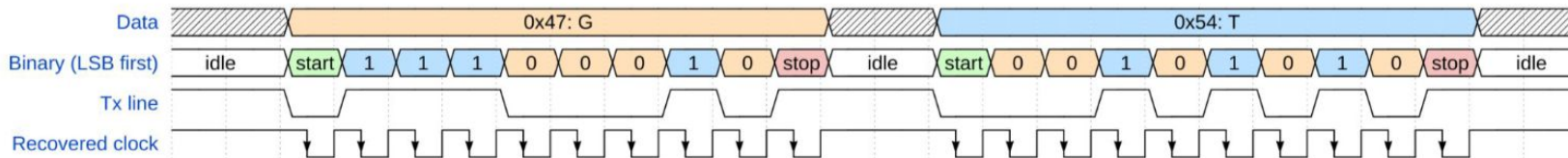
Serial / UART Universal Asynchronous Receiver Transmitter

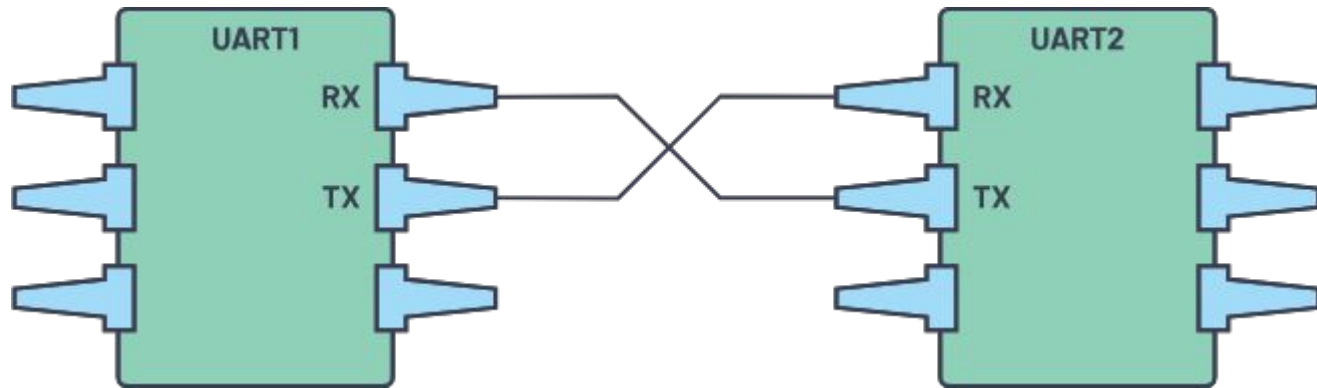
Sending "GT" at "TTL Levels" (0-5V)

Back-to-back bytes



Or with some idle time between bytes





```
// Including a system header
#include <Arduino.h>

void setup() {
    // Start the usb serial at 9600 baud
    Serial.begin(9600);
}

void loop() {
    // Send data over the usb serial port
    Serial.println("Hello World");

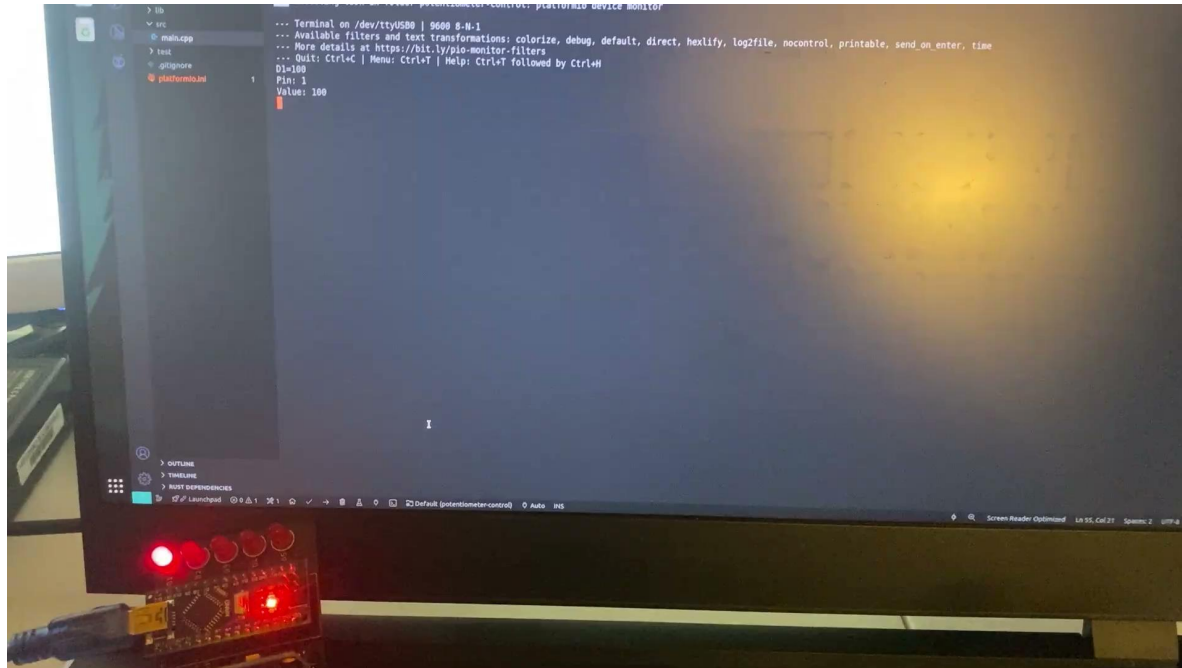
    // Receive data over the usb serial port
    if (Serial.available() > 0) {
        uint8_t data = Serial.read();
    }
}
```

Serial Communications

	SPI	I2C	Serial (async)
Typical Speed	Multiple Mbps	400 kbps	115200 bps
Typical Usage	IC to IC	IC to IC, Device to device	Device to device
Number of Devices	2 to many	2 to 127	2, only
Number of Comm Wires	$(n-1) + 3$	2	2 to 6 or more
Polarity	+	+	+ and -
Signaling	0-1.8V to 0-5V	0-1.8V to 0-5V	0-3.3V up to 15 to -15V
Pin Mode	Regular drive	Open collector	Regular drive
External components	None	2x 4.7K resistor	Maybe level converter
Distance	Inches	Feet	Yards

Labs

Serial Control



I2C IMU

